

GNU Bare-Metal Targeted Tools for Arm 32-bit Embedded Processors

NXP GCC for Arm Release 15-May-2020

Release description

Current release is based on:

- gcc 9.2.0
- binutils 2.32
- newlib 3.1.0
- gdb 8.1.1

This distribution provides support for newlib and newlib-nano.

This distribution provides tools for 32-bit host system. For 64 bit system, 32 bit libraries are required to run the tools.

Version identification

The version information (emitted via `--version` or `-v`) output format has changed to precisely define a build **BLD** identifier as an integer value whose value will always increase with each subsequent release. Example:

```
$ arm-none-eabi-gcc --version
arm-none-eabi-gcc (BLD = 1649) 9.2.0 20190812 (build.sh rev=gaf57174 s=F920 \
-i /opt/freescale Earmv7n -V release_gaf57174_build_Fed_Earmv7n)
```

```
$ arm-none-eabi-gcc -v
<snip>
gcc version 9.2.0 20190812 (build.sh rev=gaf57174 s=F920 -i /opt/freescale \
Earmv7n -V release_gaf57174_build_Fed_Earmv7n) (BLD = 1649)
```

C Libraries usage

This toolchain is released with two prebuilt C libraries based on newlib: one is the standard newlib and the other is newlib-nano for code size. To distinguish them, we rename the size optimized libraries as:

libc.a --> libc_nano.a

libg.a --> libg_nano.a

To use newlib-nano, users should provide additional gcc link time option:

`--specs=nano.specs`

Nano.specs also handles two additional gcc libraries: `libstdc++_s.a` and `libsupc++_s.a`, which are optimized for code size.

For example:

```
$ arm-none-eabi-gcc src.c --specs=nano.specs ${OTHER_OPTIONS}
```

This option can also work together with other specs options like

`--specs=rdimon.specs`

Please note that `--specs=nano.specs` is a linker option. Be sure to include it in linker options if compiling and linking separately.

Additional newlib-nano libraries usage

Newlib-nano is different from newlib in addition to the libraries' name. Formatted input/output of floating-point number are implemented as weak symbol. If you want to use %f, you have to pull in the symbol by explicitly specifying "-u" command option.

```
-u _scanf_float
```

```
-u _printf_float
```

Users can choose to use or not use semihosting by following instructions.

Semihosting

If you need semihosting, linking like:

```
$ arm-none-eabi-gcc --specs=rdimon.specs ${OTHER_LINK_OPTIONS}
```

Semihosting v2

Semihosting v2 [specification](#) introduces new extensions and the HLT encodings. Where possible, have semihosting callers continue to use the previously existing trap instructions to ensure compatibility with legacy semihosting implementations. Thus requires mixed implementation of libraries and specs.

To use new v2 specs, linking like:

```
$ arm-none-eabi-gcc --specs=rdimon-v2m.specs ${OTHER_LINK_OPTIONS}
```

Non-semihosting/retarget

If you are using retarget, linking like:

```
$ arm-none-eabi-gcc --specs=nosys.specs ${OTHER_LINK_OPTIONS}
```

User must provide its own IO subroutines (like _write, _read etc.) implementation otherwise warnings will appear on link stage:

```
undefined reference to `_write'
```

Newlib hooks

Stack pointer initialization

Original newlib stack pointer initialization has several drawbacks:

1. There is no SP initialization for Cortex-R profile cores
2. No SP initialization for different CPU modes (e.g. FIQ, ABT etc) in Thumb state
3. Different SP and SL initialization for semihosting case
4. Hardwired SP and SL values

Therefore, existing Arm crt0 code were refactored and SP and SL initialization was moved into separate `_stack_init` hook. Short description of this hook is below:

User mode only:	This routine makes default target specific Stack
+-----+ <- SL_sys,	Pointer initialization for different processor modes:
SL_usr	FIQ, Abort, IRQ, Undefined, Supervisor, System (User)

```

SYS | and setups a default Stack Limit in -case the code has
USR | --=0x10000 been compiled with "-mapcs-stack-check" for FIQ and
    | System (User) modes.
+-----+
+-----+ <- initial SP,
        | becomes SP_sys
        | and SL_usr
All modes:
+-----+ <- SL_sys,
        | SL_usr
        |
        | SYS
        | USR
        | --=0x10000
        |
+-----+ <- SP_sys,
        | SP_usr
        |
        | SVC
        | --= 0x8000
+-----+ <- SP_svc
        |
        | IRQ
        | --= 0x2000
^ +-----+ <- SP_und
s |
t | UND
a | --= 0x1000
c +-----+ <- SP_und
k |
g | ABT
r | --= 0x1000
o +-----+ <- SP_abt,
w | SL_fiq
t | FIQ
h | --= 0x1000
+-----+ <- initial SP,
        | becomes SP_fiq

```

Hard-wiring SL value is not ideal, since there is currently no support **for** checking that the heap and stack have not collided, or that this **default** 64k is enough **for** the program being executed. However, it ensures that this simple crt0 world will not immediately cause an overflow event.

We go through all execution modes and set up SP **for** each of them.

Note:

Mode **switch** via CPSR is not allowed once in non-privileged mode, so we take care not to enter **"User"** to set up its sp, and also skip most operations **if** already in that mode.

Input parameters:

- sp - Initialized SP
- r2 - May contain SL value from semihosting SYS_HEAPINFO call

Scratch registers:

- r1 - new value of CPSR
- r2 - intermediate value (in standalone mode)
- r3 - new SP value
- r4 - save/restore CPSR on entry/exit

Declared as **"weak"** so that user can write and use his own implementation **if** current doesn't fit.

```
char c2 = 2;
const char c3 = 3;
int f1 (int x) { return x; }
#pragma GCC section bss "default"
#pragma GCC section data "default"
#pragma GCC section rodata "default"
#pragma GCC section text "default"
```

Installing executables on Windows

Some executables built by MinGW requires additional libraries. Thus to support out-of-the-box execution, libwinpthread.dll comes with toolchain. This library can be moved in appropriate place if host system requires. Or MinGW package can be installed to get executables working.

Installing executables on Linux

"32-bit compatibility libraries" should be installed just in order to run 32-bit toolchains:

Ubuntu 14

```
$ sudo apt-get install lib32z1 lib32ncurses5 lib32bz2-1.0
```

Ubuntu 16

```
$ sudo dpkg --add-architecture i386
$ sudo apt-get update
$ sudo apt-get install libc6:i386 libncurses5:i386 libstdc++6:i386 lib32z1
```

Debian

```
$ sudo apt-get install lib32z1 lib32ncurses5 lib32stdc++6
```

CentOS

```
$ sudo yum install glibc.i686 ncurses-devel.i686
```

GDB with Python

Additionally if you want to use gdb python build (arm-none-eabi-gdb-py) on Linux, you need to install 32 bit python2.7 and compatibility libraries.

Ubuntu

```
$ sudo apt-get install libpython2.7:i386
$ sudo apt-get install python2.7-minimal:i386 python2.7:i386 python2.7-dev:i386
```

And after that switch default python to new one:

```
$ sudo ln -s usr/bin/python2.7 python
```

CentOS

```
$ sudo yum install python-libs.i686
```

Windows

To use gdb python build (arm-none-eabi-gdb-py.exe) on Windows, you need to install 32 bit python2.7 no matter 32 or 64 bit Windows. Please get the package from <https://www.python.org/download/>.

Release history

Date	Description
29-January-2020	Initial release
bld=1649 rev=gaf57174	Single precision and NEON libraries for Cortex-r52 (armv8-r) added